Web Development Fall 26 Project

Choose one of the project ideas

Project Idea 1: Reddit Clone

Introduction

The project is to build a minimal Reddit clone. Reddit is a social media platform where people can:

- Join/create communities
- create posts
- comment and upvote other people's posts

Check out reddit.com

Now reddit is a very complicated platform with many many functionalities. In this project we will try to create the basic requirements of the platform and any extra features added will be rewarded with bonus marks.

Requirements

The UI must match the actual reddit UI as closely as possible.

Basic requirements:

1. Account creation and login
2. Viewing/Editing user's profile
3. Creating a community
4. Joining/Leaving a community
5. Creating a post on a community
6. Viewing posts of a community
7. Viewing feed page
8. Upvoting/Downvoting posts
9. Commenting on posts
10. Searching for communities and users
11. Replying to comments (nested comment threads)
12. Sorting posts (for example New and Top) and loading them in pages on the feed and community pages
13. Editing and deleting your own posts and comments
14. Community moderation: the creator of a community is its moderator and can remove posts and comments in that community

Voting rule: each user can have at most one vote on a post, and switching or removing a vote must update the score correctly, even when several votes are submitted at the same time. Your backend design will be assessed on how it handles this case.

AI integration feature

We will be adding a new feature where users have the option to summarize a post using generative AI. Don't panic! It's very simple to implement.

You can get inspiration for extra features by checking reddit itself.

Project Idea 2: Event Ticketing Platform

Introduction

The project is to build an event ticketing platform where there are organizers that create events and sell tickets for them, and attendees that browse events and book tickets.

The platform will work as follows:

1. Organizers create events and define the ticket types and quantities available for each event
2. Attendees browse events and book tickets
3. Organizers receive the bookings, manage their events, and check attendees in on the day of the event

Requirements

Since this is not a clone, you will have to design your UI.

You will need to develop 2 separate web apps: one for the organizers and one for the attendees.

Real payment is out of scope. Booking is completed with a simulated checkout (no payment gateway integration is required).

Capacity rule: the system must never sell more tickets than the capacity defined by the organizer, even when several attendees try to book the last remaining tickets at the same time. Your backend design will be assessed on how it handles this case.

Since showing up and running the event are the responsibility of the attendee and the organizer respectively, the system will work using a flagging system. The attendees can flag organizers that cancel events without notice or misrepresent them, and the organizers can flag attendees that book tickets and do not show up.

Adding QR-code check-in (each ticket carries a unique QR code that the organizer scans to check the attendee in) will be rewarded with bonus marks.

Organizer app requirements:

1. Account creation and login
2. Creating events (title, description, category, date and time, venue, image)
3. Defining ticket types for each event (name, price, quantity), for example General and VIP
4. Editing and cancelling events
5. Viewing the bookings and the attendee list of each event
6. Checking in attendees (marking tickets as used)
7. Flagging attendees

Attendee app requirements:

1. Account creation and login
2. Viewing the events page
3. Searching for events
4. Filtering events by category and date
5. Booking tickets
6. Viewing and cancelling bookings
7. Rating events
8. Adding comments on events
9. Flagging organizers

AI integration feature

Attendees can get a summary of the comments added for an event.

Project Idea 3: Team Task Management (Trello Clone)

Introduction

The project is to build a minimal Trello clone. Trello is a task management platform where teams can:

- Create boards to organize their work
- Organize tasks as cards inside lists (for example To Do, In Progress, Done)
- Assign, discuss, and move tasks between lists as the work progresses

Check out trello.com

Now trello is a very complicated platform with many many functionalities. In this project we will try to create the basic requirements of the platform and any extra features added will be rewarded with bonus marks.

Requirements

The UI must match the actual trello UI as closely as possible.

Basic requirements:

1. Account creation and login
2. Viewing/Editing user's profile
3. Creating a board
4. Inviting members to a board and removing them
5. Creating, editing, and deleting lists on a board
6. Creating, editing, and deleting cards in a list
7. Moving cards between lists and reordering them using drag and drop
8. Assigning members to cards
9. Setting due dates and labels on cards
10. Commenting on cards
11. Searching for boards and cards
12. Real-time updates: changes made by one member (cards, lists, comments, moves) appear for the other members viewing the same board without refreshing the page (for example using WebSockets)

Access rule: a board must only be visible to its members, and only the board owner can delete the board or manage its members.

Adding Jira-like features (for example issue types such as bug, story, and task; a backlog with sprints; or configurable workflow statuses with rules about which moves are allowed) will be rewarded with bonus marks.

AI integration feature

Users have the option to summarize the comment thread of a card using generative AI.

You can get inspiration for extra features by checking trello itself.

Deliverables

1. Codebase on GitHub.
2. Database schema diagram.
3. APIs documentation.
4. Contributions of each team member. If you have your name as a contributor on a point, you will be asked about it in the discussion.

Teams

Students may work in groups up to 6 students per group.

Timeline

This timeline is recommended but students are free to work as they see fit.

Weeks	Work
3-5	• Creating the teams • Requirements analysis: breaking down the requirements into user stories. • Database design.
6	• Frontend
7	• Midterms
8	• Frontend • Backend design
9-12	• Frontend • Backend • Integration
13-14	• Extra Features • Documentation • Deployment
15	• Discussions

Tech stack

Students are free to work with the tech stack of their choice. In our labs we will be using MERN stack.

Grading Criteria

Team grades:

Item	Weight
Basic Requirements	25%
AI integration requirement	10%
UI	15%
Database Design	5%
APIs Design: use swagger, postman or any tool for API documentation	10%
Clean Code	5%

Reddit bonus items

Bonus Item	Weight
Chatting feature	5%
Deployment	5%

Event ticketing bonus items

Bonus Item	Weight
QR-code check-in	5%
Deployment	5%

Trello clone bonus items

Bonus Item	Weight
Jira-like features (issue types, backlog and sprints, configurable workflow statuses)	5%
Deployment	5%

- Any extra features added will be rewarded 2 extra marks per feature.
- Deployment means that the project is available for use and can be accessed by any device from anywhere.
- Note that the team grade including the bonus is capped at 80% of the project mark. Meaning that the team grade component cannot get higher than 80% of the project grade.

Individual Grades

Item	Weight
Discussion	30%

Academic Integrity

Use of Generative AI (e.g., ChatGPT) is permitted, but must be used responsibly.

Students are expected to fully understand all aspects of the code they submit. During the project discussion, if a student is unable to explain parts of their own code, they will be deducted marks. Be prepared for an in-depth discussion.

Plagiarism (copying someone else’s code) is prohibited.